

Deadline. The Project is due **11:59pm, Mon 05/04, 2026**.

Submit Your Work. You must submit your codes via GitHub, and submit your report via GradeScope. In the test, we will only use the header file (`quicksort.h`). All functions you implemented must be in that header file. In our final test, we will call the functions defined in that file. See more details below.

Late Policy. You have up to two grace days (calendar day) for the entire quarter. You don't lose any point by using grace days. If you decide to use grace days, please specify how many grace days you would like to use and the reason at the beginning of your submission.

Academic Integrity. You can discuss with your classmates or use online resources. In particular,

- **Collaboration Policy.** You can discuss your implementations and your optimizations with your classmates. You can get help from the instructor, but only after you have thought about the problems on your own. If you discussed with anyone else, you need to acknowledge them in your report.
- **Other sources.** It is OK to get inspiration (but not solutions) from books or online resources, again after you have carefully thought about the problems on your own. If you use any reference or webpage, or discussed with anyone, you must cite it fully and completely in your report.
- **Other Libraries.** You are free to use any sequential code in C++ standard library (e.g., `std::sort`) by including the corresponding header files. However, you cannot use any existing parallel libraries. In particular, all codes you submitted from `quicksort.h` must be implemented on your own. You are free to use any previous code implemented by yourself. For example, if you have implemented some code in project 1, you can directly copy them to project 2 if you need.

However, you **cannot**:

- Share your report/code with anyone else.
- Read other's report/code.
- Copy anything from other source, including code and report.
- Get help from others without acknowledging them in your report, or use any relevant resources (e.g., online article) without citing them.

Otherwise it will be considered cheating. We reserve the right to deduct points for using such material beyond reason.

Write-up. You need to submit a report describing your methodology, optimizations, and possible experiments you designed to test the performance and the experimental results. The report will be an important part in grading.

In this project, we will implement a parallel sorting algorithm as we talked about in class.

1 Overview

You can find the template files by this invitation link

https://classroom.github.com/a/5fyyA_mv

The repository includes the following files

- **quicksort.cpp**. This file is an example testing file for you. The actual testing code will be very similar, but differs in how to generate input data. As you will see, the only function you need to implement is the **quicksort** function. When you test your own code, you can modify any files as you need, but please don't leave anything useful in this cpp file. You can implement your distributions in this file.
- **quicksort.h**. You need to implement your sorting function in this file. Of course you can add auxiliary functions or variables, but please make sure to put all your code in this file.
- **Makefile**. This is also an example **Makefile** to help you test your code. It is basically the same as what we will use in final test. You can edit the file as you like. Also, do not put anything useful in this file since that's not used in the final test.

Although the files are named "quicksort", you can use **any sorting algorithms** and add any optimization you want (i.e., it doesn't need to be a standard quicksort).

2 Brief Guideline

Here's a brief guideline.

1. For the best performance, you probably want to avoid allocating memory during the algorithm. That is to say, if you need extra space, allocate the memory in advance. This is because allocating memory in parallel can be expensive.
2. Don't forget to consider granularity control for all building blocks in your algorithm.
3. When you need to generate random numbers, do not use the default one. In class we mentioned that you can just generate pseudo random numbers by using a hash function with random seeds.
4. You may want to test the correctness of your algorithm, e.g., you can use STL sorting algorithm compare your output with STL.
5. Finally, you need to test the performance of your code. Try to design different test cases and test their performance. If they show different performance, try to think about the reason behind that.

3 Possible Optimizations

Here are some optimizations and hints you can consider in your implementation. Of course, you can use more.

1. Coarsening. Use the similar idea as shown in Homework 0 and 1. Try to find the best threshold to determine a sequential or a parallel run.
2. You can call STL sorting algorithm or other existing sorting algorithms to work on the base cases. Since STL is highly-optimized, it's likely to give better performance than your sequential version.

3. Try to avoid using `std::vector`, especially avoid resizing it in parallel. This is because resizing involves allocating and deallocating memory, which is also slow in parallel.
4. Try to design a hash function to generate random numbers instead of `rand()`. This is because this function is not designed for running in parallel.
5. Choosing an arbitrary key as the pivot gives good theoretical bound using randomization, but in practice we can even do more to avoid bad luck. For example, you can pick up t random elements in the original array, find the median of them, and use the median as the pivot. This helps to split your input in a more balanced way. t is usually selected as a small constant.
6. Of course you can think of more interesting optimizations!

4 Design Your tests

To test the performance, you will need to design some test cases with different patterns. This is to make sure your algorithm performs well in different inputs, and to understand the good and bad cases for your algorithm. In our final test, we will have five test cases, each of size 10^8 . In all test cases, the input keys are 64-bit integers. For each test, we will use the average of the five runs as your final running time (running four rounds, and report the average of the last three rounds). The actual testing data will not be given (you need to design your own input to test and debug), but the data pattern will be shown below.

Test case	Size	Data
1	10^8	Mostly distinct numbers, integers, uniformly random
2	10^8	Moderate duplicates, skewed distribution
3	10^8	Heavy duplicates, uniformly random
4	10^8	Heavy duplicates, skewed distribution
5	10^8	Real numbers, unknown distribution

If you want to estimate the performance, you can assume the five distributions to be **similar to**:

1. Test 1: all elements are uniformly at random in range $[0, 10^9]$.
2. Test 2: all elements are from a Zipfian distribution.
3. Test 3: all elements are uniformly at random in range $[0, 100]$.
4. Test 4: all elements are from a Zipfian distribution with similar level of duplication as Test 3.
5. Test 5: all elements are real numbers from in $[0, 1]$. The distribution is arbitrary (not evenly distributed), e.g., certain ranges can be denser than others.

If you use comparison-based sort, the skewness should not make much difference to the performance. Therefore our grading basis is also the same for Tests 1, 2, and 3, and the same for Tests 4 and 5.

The final tests are **not exactly the same as stated above**. But they will be reasonably similar. For example, every element will be added the same offset, or multiplied by a constant, etc. This is just to avoid specific optimizations that overfit the tests above. Your goal is to design general algorithms that work well for any inputs.

5 Grading

Your grade will consist of two parts: final report (5pts), performance (10pts + bonus, see more details below).

For grading, we will test five different input instances. Each of them will have different data distribution, heavy/light duplications, different data types (integer/real numbers) etc. Your goal is to make your algorithm robust for all different data distributions.

To get the running time point, your algorithm has to be **correct**. That means, if your output is wrong, you cannot get points for the corresponding tests.

At the end of Week 5, we will release a set of example tests along with the grading standard for this project. You can first focus on the algorithm implementation, general test and correctness in the first week.

In the report, you need to describe your methodology, optimizations and their performance, possibly with running time data.

Note that the report takes 1/3 of the entire project grade. They are supposed to be formal and in details.

6 Useful Resources

Although sorting seems to be a basic operation, it is still heavily studied in recent research work. Here I provide a few state-of-the-art sorting-related papers and open-source libraries. You can take a look at their paper/code if you are interested, and probably get some more inspirations for your own implementation.

- ParlayLib sorting [2]. The ParlayLib is a library with a collection of parallel primitives. Our scheduler for the class is also taken from a previous version of ParlayLib. You can find its documentation here, and the code here. Under `include/parlay/internal`, you will find a few algorithms for sorting, including merge sort, quick sort, sample sort, bucket sort, integer sort, etc. The sample sorting algorithm is mostly based on the theoretical results in [3], but with a lot of implementation improvements for better performance in practice.
- IPS⁴o [1]. An efficient parallel sample sort algorithm. The advantage of this algorithm is that it's in-place, i.e., it avoids using much auxiliary memory to improve performance. The code is available here. The same paper also describes an integer sort version IPS²Ra algorithm available here.
- There are also relevant work on integer sort [4] and semisort [5]. They are not designed for general sorting, but they are more recent for a summary of related work for parallel sorting, as well as some figures for an introduction of some general techniques for sorting. You can use them as additional reading if you want.

In your project, we do not allow you to directly copy code from these libraries, but you can feel free to learn and use the ideas in these existing works. In particular, all these libraries are highly optimized with their own infrastructure to accelerate sorting. You cannot use these libraries by directly copying or including them. However, you can implement similar algorithms in your own `.h` file.

References

- [1] Michael Axtmann, Sascha Witt, Daniel Ferizovic, and Peter Sanders. Engineering in-place (shared-memory) sorting algorithms. *ACM Transactions on Parallel Computing*, 9(1):1–62, 2022.
- [2] Guy E Blelloch, Daniel Anderson, and Laxman Dhulipala. Parlaylib-a toolkit for parallel algorithms on shared-memory multicore machines. In *Proceedings of the 32nd ACM Symposium on Parallelism in Algorithms and Architectures*, pages 507–509, 2020.

- [3] Guy E Blelloch, Phillip B Gibbons, and Harsha Vardhan Simhadri. Low depth cache-oblivious algorithms. In *Proceedings of the twenty-second annual ACM symposium on Parallelism in algorithms and architectures*, pages 189–199, 2010.
- [4] Xiaojun Dong, Laxman Dhulipala, Yan Gu, and Yihan Sun. Parallel integer sort: Theory and practice. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 301–315, 2024.
- [5] Xiaojun Dong, Yunshu Wu, Zhongqi Wang, Laxman Dhulipala, Yan Gu, and Yihan Sun. High-performance and flexible parallel algorithms for semisort and related problems. In *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures*, pages 341–353, 2023.